

① Dynamic Programming :- It is used

to solve 2 type of problems :-

1. Finding optimal solution for any problem (min-max value)
2. Count no of ways in which certain stuff which is asked in the problem could be achieved.

① Any question related to D-P can be solved using 2 ways :-

1. Top-Down Approach
(Recursion + Memorization)

2. Bottom-Up Approach :- In this approach, you used 1-D, 2-D array and start filling it from base index i.e 0 or (0,0).

eg:- $f(n) = f(n-1) + f(n-2)$

$$f(4) = f(3) + f(2) = 2 + 1 = 3$$

$$f(3) = f(2) + f(1) = 2$$

$$f(2) = f(1) + f(0) = 1$$

$$f(1) = 1$$

$$f(0) = 0$$

(Base Condⁿ)

∴ Base Condⁿ =

$$f(5) = \underbrace{f(4)}_2 + \underbrace{f(3)}_2$$

① Top - Down Approach

Recursion

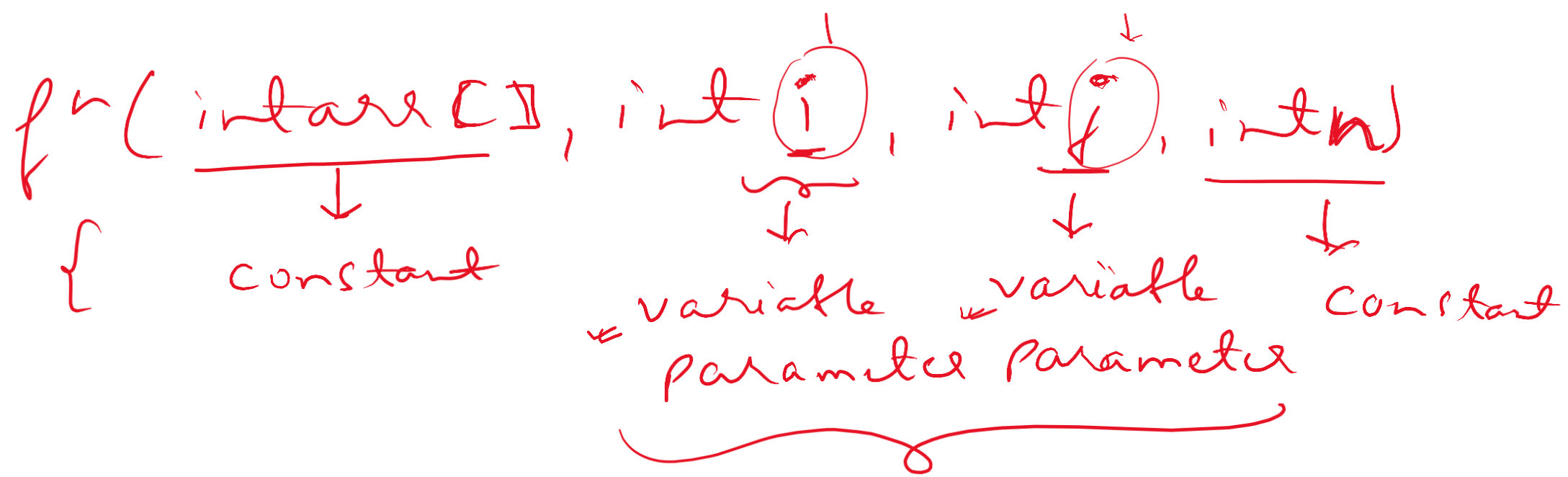
1. Parameters
2. Base Condⁿ
3. Make Recursive
Fn call by changing
parameters
4. Return correct value
back to parent's call.

+

Memorization

1. HashMap (key, value).
 2. 2-D - array
 3. 1-D - array
- ① If we are using (Key, value)
pair, Key $\Rightarrow$ unique
for each & every subproblem.

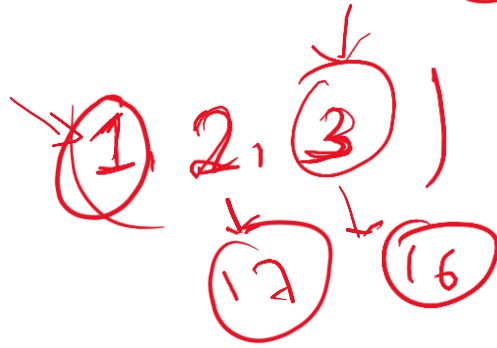
①



String Key = $i + " , " + j$

$\hookrightarrow$ dp[m][k]

1. target sum, $\Rightarrow$ 19 18
2. coins C
3. N



Index

int f (coins C, target sum, N, int[] dp)

Annotations:

- coins C: constant
- target sum: variable parameter
- N: variable parameter
- int[] dp: dp array

if (target sum == 0) return dp[N] (target sum = 19)

if (n == 0) return 0,

if (dp[n][targetSum] != -1)
return dp[n][targetSum];

coins = [2, 1]

if (coins[n-1] <= targetSum)
// Picking coin
{
dp[n][targetSum] = fⁿ(coins, targetSum, -coins[n-1],
return // not picking coin. n, dp)
+ fⁿ(coins, targetSum, n-1, dp);
}

else

{

return $dp[n][\text{target sum}] = f^n(\text{coins}, \text{target sum}, n-1, dp)$;

}

}